



```
In [ ]: DAY 27 CODING CHALLENGE
```

```
In [ ]:
```

```
In [2]: trips = (  
        (101, "Bangalore", 12.5, 250),  
        (102, "Hyderabad", 8.0, 180),  
        (103, "Bangalore", 15.2, 300),  
        (104, "Chennai", 5.5, 120),  
        (105, "Bangalore", 7.8, 200)  
    )  
  
    total_trips = len(trips)  
    print("Total Trips:", total_trips)
```

Total Trips: 5

```
In [3]: bangalore_trips = sum(1 for trip in trips if trip[1] == "Bangalore")  
    print("Trips from Bangalore:", bangalore_trips)
```

Trips from Bangalore: 3

```
In [4]: total_revenue = sum(trip[3] for trip in trips)  
    print("Total Revenue:", total_revenue)
```

Total Revenue: 1050

```
In [5]: print("High Distance Trips (>10 km):")  
    for trip in trips:  
        if trip[2] > 10:  
            print(trip)
```

High Distance Trips (>10 km):
(101, 'Bangalore', 12.5, 250)
(103, 'Bangalore', 15.2, 300)

```
In [6]: highest_fare_trip = max(trips, key=lambda trip: trip[3])  
    print("Highest Fare Trip:", highest_fare_trip)
```

Highest Fare Trip: (103, 'Bangalore', 15.2, 300)

```
In [7]: bangalore_fares = [trip[3] for trip in trips if trip[1] == "Bangalore"]  
    avg_fare = sum(bangalore_fares) / len(bangalore_fares)  
  
    print("Average Fare (Bangalore):", avg_fare)
```

Average Fare (Bangalore): 250.0

```
In [ ]:
```

```
In [ ]: ## Brief explanation Regarding Ride booking
```

```
In [ ]: This analysis examines ride-booking trip data stored as tuples, where each tri  
Basic operations such as counting trips and filtering by city help understand  
Revenue analysis is performed by summing fares, while identifying high-distance
```

The highest fare trip indicates premium rides, **and** average fare calculations r
Overall, this exercise demonstrates how simple data structures can be used **for**